Universidade Estadual de Campinas – UNICAMP
Instituto de Matemática, Estatística e Computação Científica

Relatório – Projeto 1: Reconhecimento de Objetos Usando YOLO

Disciplina: Introdução ao Aprendizado de Máquinas Profundo

Professor: João Florindo

Alunos:

Amanda Rehder Cedro	**RA:** 253507
Elisa Amorim Da Costa	**RA:** 277145
Iuri Santos Oliveira	**RA:** 194610
Pedro Galhardo Barbosa	**RA:** 277218

7 de maio de 2026

Conteúdo

1 Introdução ao Problema de Detecção de Objetos **2**
1.1 Definição do Problema . . . 2
1.2 Importância . . . 2
1.3 Principais Métodos Atuais . . . 2

2 O Algoritmo YOLO — Arquitetura e Funcionamento **3**
2.1 Intuição e Motivação . . . 3
2.2 Funcionamento Passo a Passo . . . 3
2.3 Arquitetura do YOLO v11 . . . 4

3 Resultados **5**
3.1 Configuração Experimental . . . 5
3.2 Detecções Base Fornecida (conf=0.5, IoU=0.5) . . . 5
3.3 Efeito da Variação de Limiares . . . 5
3.3.1 Os Limiares de Decisão: Confiança e IoU . . . 5
3.3.2 Análise prática a partir de imagens do dataset original da disciplina 6
3.3.3 Teste com Imagens Externas . . . 9

4 Discussão **11**
4.1 Qualidade Geral e Papel dos Limiares . . . 11
4.2 Comportamento do NMS por Classe . . . 11
4.3 Generalização em Imagens Externas . . . 11
4.4 Limitações e Propostas de Melhorias . . . 12

5 Referências **13**

1 Introdução ao Problema de Detecção de Objetos

1.1 Definição do Problema

A detecção de objetos é uma das tarefas fundamentais em Visão Computacional. Diferente da simples classificação de imagens, que responde apenas "o que há na imagem?", a detecção de objetos responde simultaneamente "o que há?" e "onde está?", localizando cada objeto por meio de uma *bounding box* (caixa delimitadora) e atribuindo uma classe a ele.

Formalmente, dado uma imagem de entrada, o objetivo é produzir uma lista de tuplas $(c, p, b_x, b_y, b_w, b_h)$, onde c é a classe do objeto, p é a probabilidade de confiança, e (b_x, b_y, b_w, b_h) definem a bounding box.

1.2 Importância

A detecção de objetos é o núcleo da compreensão de cena. Sem ela, a inteligência artificial seria capaz de vizualizar o ambiente, mas incapaz de interpretar e agir sobre os elementos do espaço. É essencial em diversas aplicações modernas, como por exemplo:

- **Veículos autônomos:** Detecção de pedestres, veículos, semáforos e placas em tempo real para navegação segura.
- **Vigilância e segurança:** Monitoramento automático de câmeras para detecção de intrusos.
- **Medicina:** Detecção de tumores e anomalias em imagens médicas.
- **Varejo e indústria:** Controle de qualidade e contagem de produtos em prateleiras.

1.3 Principais Métodos Atuais

Os métodos de detecção podem ser divididos em duas categorias:

- **Métodos de dois estágios (Two-stage):** R-CNN, Fast R-CNN, Faster R-CNN. Propõem regiões candidatas e depois as classificam. São mais precisos, porém mais lentos.
- **Métodos de um estágio (Single-stage):** YOLO, SSD, RetinaNet. Tratam a detecção como um problema de regressão único, processando a imagem inteira de uma vez. O YOLO destaca-se por sua velocidade para aplicações em tempo real.

2 O Algoritmo YOLO — Arquitetura e Funcionamento

2.1 Intuição e Motivação

O nome "You Only Look Once" resume a ideia: em vez de examinar a imagem múltiplas vezes (como a R-CNN), o YOLO processa a imagem inteira em uma única passada, tratando a detecção como um **problema de regressão** direto para coordenadas e probabilidades.

2.2 Funcionamento Passo a Passo

Passo 1: Divisão em Grade

A imagem é dividida em uma grade $S \times S$. Cada célula detecta objetos cujo ponto médio esteja dentro dela.

Sem a grade, a rede ficaria "perdida" tentando prever coordenadas soltas. Com a grade, o problema de encontrar um objeto em qualquer lugar da imagem se transforma em S^2 pequenos problemas de encontrar um objeto relativo ao centro de cada célula.

Passo 2: Predição por Célula

Cada célula prevê a probabilidade do objeto (p_c), as coordenadas relativas (b_x, b_y, b_w, b_h) e as probabilidades condicionais das classes ($c_1, \ldots, c_n$).

A motivação por trás da predição por célula é a unificação de tarefas. Cada célula da grade é responsável por prever um vetor que contém a probabilidade de existência de um objeto (P_c), as coordenadas da *bounding box* e a distribuição de probabilidade das classes.

A intuição principal é o uso de coordenadas relativas à célula: os valores de centro (b_x, b_y) são previstos como deslocamentos entre 0 e 1 em relação aos limites da própria célula. Essa estratégia simplifica o espaço de busca da rede neural, tornando o treinamento mais estável e permitindo que o modelo realize a classificação e a localização em uma única operação tensorial, o que é a chave para o desempenho em tempo real do algoritmo.

Passo 3: Anchor Boxes

Utiliza-se formas pré-definidas representando diferentes proporções (ex: vertical para pessoas, horizontal para carros) para permitir múltiplos objetos numa mesma célula.

A motivação para o uso de *Anchor Boxes* (caixas de âncora) é resolver a limitação de detectar múltiplos objetos que compartilham a mesma célula da grade e possuem proporções distintas. A intuição baseia-se em fornecer ao modelo "moldes" pré-definidos de larguras e alturas, calculados via agrupamento *K-Means* sobre o conjunto de treinamento. Em vez de prever dimensões absolutas, a rede aprende a ajustar (*offsets*) estas âncoras para melhor envolver o objeto.

Isso permite que uma única célula se especialize: uma âncora pode focar em objetos verticais (como pedestres), enquanto outra foca em horizontais (como veículos), permitindo a detecção simultânea de objetos sobrepostos com maior precisão geométrica.

Passo 4: Intersection over Union (IoU)

O IoU, também conhecido como Índice de Jaccard, é a métrica padrão utilizada para quantificar a precisão da localização de um objeto. Em problemas de detecção, o modelo prevê uma caixa (B_p) e nós temos a caixa real ou ground truth (B_{gt}). O IoU mede quão sobrepostas essas duas caixas estão.

Avalia a qualidade preditiva comparada ao *ground truth*:

$$IoU = \frac{\text{Área da Interseção}}{\text{Área da União}} \tag{1}$$

O resultado do IoU varia estritamente entre 0 e 1:

IoU = 0: Não há qualquer sobreposição entre as caixas.

IoU = 1: As caixas estão perfeitamente alinhadas (predição idêntica ao real).

0.5 $\leq$ IoU: Geralmente considerado uma "boa" detecção em competições como o PASCAL VOC.

0.75 $\geq$ IoU: Considerado uma detecção de alta precisão (rigorosa).

Passo 5: Non-Max Suppression (NMS)

Elimina detecções redundantes independentemente para cada classe:

1. Descartar caixas com confiança abaixo do limiar.
2. Selecionar a caixa de maior confiança da classe em questão.
3. Remover caixas da mesma classe com $IoU \geq$ limiar em relação à selecionada.
4. Repetir o processo.

A motivação de fazer o NMS de forma independente para cada classe é evitar que objetos diferentes, mas muito próximos, se apaguem. Por exemplo, se uma pessoa está segurando um guarda-chuva, as caixas estarão muito sobrepostas. Se o NMS não fosse por classe, a caixa da pessoa (com maior confiança) poderia apagar a caixa do guarda-chuva.

2.3 Arquitetura do YOLO v11

Utilizamos o **YOLO v11 nano** (yolo11n), com **238 camadas** e **2.583.258 parâmetros**. A saída gera predições em 3 escalas diferentes. Destacam-se as camadas C3k2 (extração no Backbone), SPPF (agregação multi-escala) e C2PSA (mecanismo de atenção).

3 Resultados

3.1 Configuração Experimental

- **Modelo:** YOLO v11 nano (yolo11n.pt) pré-treinado no dataset COCO.
- **Implementação:** Python (Ultralytics) com IoU e NMS implementados manualmente de forma independente por classe.
- **Imagens testadas:** 12 imagens da base fornecida + 4 imagens externas.
- **Limiares testados:** Confiança $\in \{0.3, 0.5, 0.7\}$, IoU $\in \{0.3, 0.5, 0.7\}$.

3.2 Detecções Base Fornecida (conf=0.5, IoU=0.5)

Alguns exemplos de saída do código que podem ser observadas são:

Imagem	Objetos	Classes Detectadas	Conf. Máxima
cat.jpg	1	cat	63.0%
city_scene.jpg	8	person, traffic light, car, truck	75.6%
dog.jpg	2	dog, bicycle	92.1%
horses.jpg	5	horse (×5)	92.9%

3.3 Efeito da Variação de Limiares

3.3.1 Os Limiares de Decisão: Confiança e IoU

Para que o modelo YOLO entregue resultados práticos, ele depende de dois filtros fundamentais, que determinam o que será exibido como uma detecção válida e o que será descartado como ruído. O ajuste desses limiares (Confiança e IoU) é o que define o comportamento do modelo em diferentes cenários.

Começando pelo Limiar de Confiança, ele funciona como o primeiro grande filtro do modelo. Cada caixa (*bounding box*) sugerida pelo YOLO vem acompanhada de um valor probabilístico que indica o grau de certeza da rede de que ali existe um objeto e de que a caixa está bem posicionada. Por exemplo, se o limiar for definido em 0.5, qualquer detecção com menos de 50% de certeza é ignorada.

Em termos práticos, um limiar baixo fará com que o modelo encontre quase todos os objetos da cena; porém, isso gera muitos falsos positivos, já que haverá a detecção de objetos que não existem, confundindo texturas ou sombras com classes reais. Em contrapartida, um limiar alto aumenta a precisão, pois o modelo só mostra aquilo de que tem muita certeza. Contudo, isso pode causar falsos negativos, ignorando objetos reais que são difíceis de identificar, como algo ao fundo da imagem ou mal iluminado.

Já o limiar do IoU é uma métrica que mede a sobreposição entre duas caixas. Ele serve para "limpar" a imagem de detecções duplicadas. Assim, como o YOLO pode prever várias caixas para um mesmo carro, por exemplo, o NMS usa o IoU para manter apenas a melhor.

Dessa forma, quando temos um limiar de IoU baixo, o filtro fica "muito agressivo"; logo, se as caixas se sobrepuserem apenas um pouco, o modelo já elimina uma delas, o que é ruim para cenários de alta densidade. Já em um limiar de IoU alto, o filtro permite com mais facilidade que caixas fiquem muito sobrepostas sem serem excluídas, sendo ideal para detectar objetos que estão colados uns aos outros, mas pode acabar deixando a imagem poluída com várias caixas repetidas para um único objeto, se o modelo não for preciso.

3.3.2 Análise prática a partir de imagens do dataset original da disciplina

A seguir, apresentamos a aplicação do modelo YOLO v11 sobre diferentes cenários para avaliar o desempenho do algoritmo sob variações nos limiares de decisão. O objetivo é observar empiricamente como os limiares de confiança e IoU discutidos anteriormente afetam a capacidade de generalização, a precisão na localização e a contagem total de objetos em cenas com diferentes níveis de complexidade, densidade e iluminação. Cada imagem a seguir foi acompanhada por uma tabela para ajudar no entendimento do assunto.

Na Figura 1, é possível perceber, nas primeiras imagens que possuem configurações de baixa confiança (Conf=0.3), que o YOLO consegue capturar as pessoas ao fundo; contudo, a variação do IoU de 0.7 para 0.3 revela a atuação do NMS, que se torna um pouco mais agressivo ao sumir com detecções sobrepostas, reduzindo a contagem de 6 pessoas (8 objetos) para 5 pessoas (7 objetos).

Figura 1: Análise comparativa de detecção na imagem de controle (cachorro).

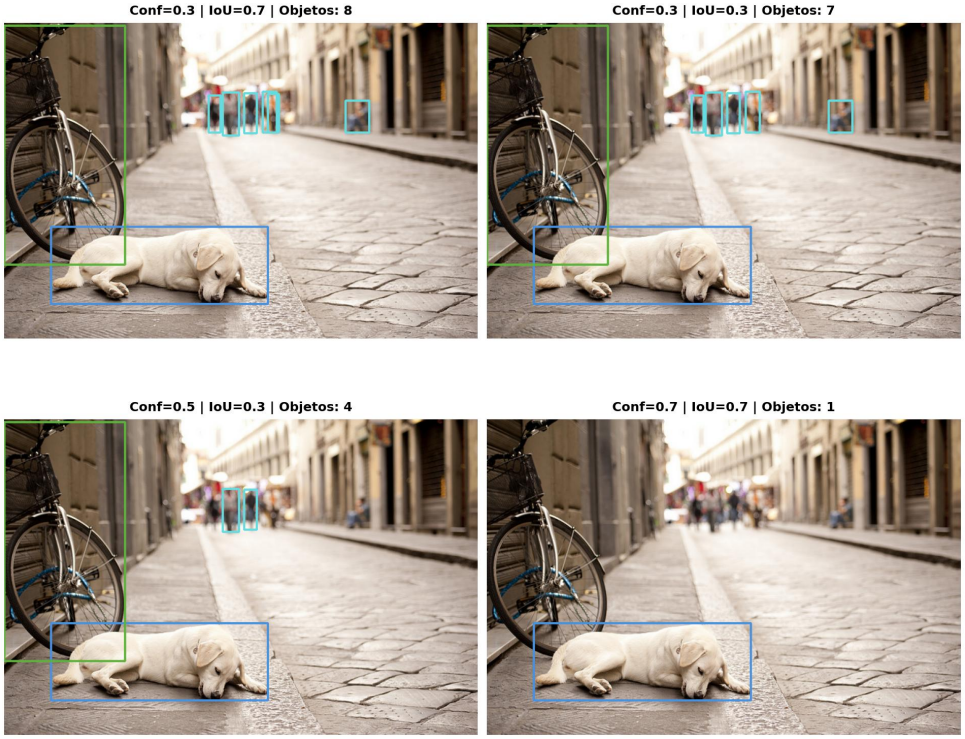


Tabela 1: Comparação de limiares para a imagem de um cachorro

Conf	**IoU**	**N. Obj**	**Classes Detectadas**
0.3	0.7	8	dog, person, bicycle, person, person, person, person, person
0.3	0.3	7	dog, person, bicycle, person, person, person, person
0.5	0.3	4	dog, person, bicycle, person
0.7	0.7	1	dog

Em contraste, o aumento do limiar de confiança para 0.5 e depois para 0.7 resulta em uma queda na detecção. Isso acontece porque o modelo possui baixa convicção sobre os objetos em escala reduzida ou parcialmente excluídas, priorizando a precisão em detrimento da completude.

A alta densidade e a sobreposição dos alimentos na Figura 2 aumentam sua complexidade em relação à imagem anterior (Figura 1). Isso faz com que um limiar de confiança baixo (Conf=0.3) e um IoU permissivo (0.7) gerem uma contagem elevada de 23 objetos, em que muitos são detecções redundantes ou fragmentações de um mesmo prato. Quando reduzimos o IoU para 0.5, observa-se uma atuação ainda mais rigorosa do algoritmo de *Non-Max Suppression* (NMS), que consegue eliminar duplicatas espaciais, limpando o excesso de caixas sobrepostas que tentavam classificar a mesma porção.

Figura 2: Análise de detecção em cena com pratos de comida.

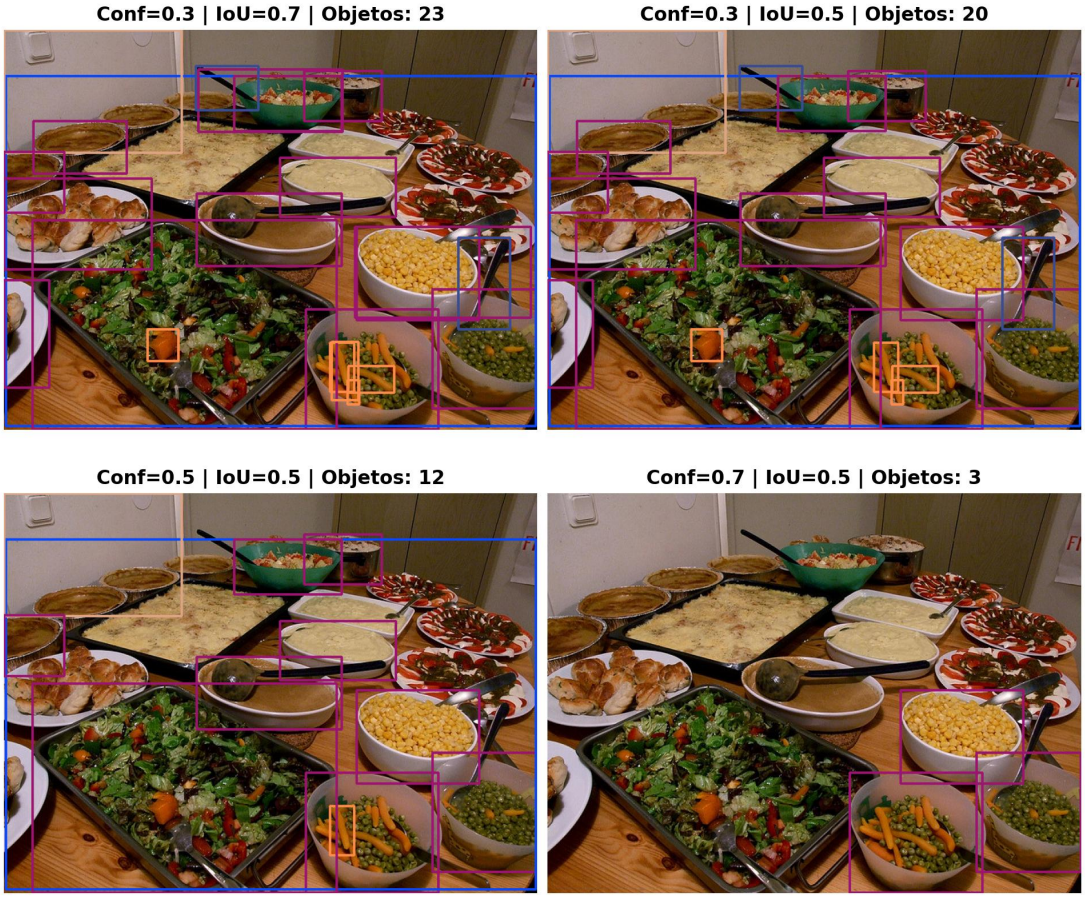


Tabela 2: Comparação de limiares em pratos de comida

Conf	IoU	N. Obj	Classes Detectadas
0.3	0.7	23	bowl (x14), diningtable, refrigerator, carrot (x5), spoon (x2)
0.3	0.3	20	bowl (x12), diningtable, refrigerator, carrot (x4), spoon (x2)
0.5	0.3	12	bowl (x9), diningtable, refrigerator, carrot
0.7	0.7	3	bowl (x3)

A mudança para limiares de confiança mais altos nos mostra que o modelo, em ambientes com texturas orgânicas e variadas, fica mais frágil. Como, por exemplo, no momento em que houve um aumento de Conf = 0.3 para Conf = 0.5, em que tivemos uma queda significativa; ali, perde-se a certeza sobre diversos itens. Esse efeito alcança o seu ápice quando o limiar de confiança sobe para 0.7, e a detecção é reduzida a apenas 3 objetos, demonstrando que uma exigência por alta precisão acaba por omitir dados. Em comparação com a análise anterior, nota-se um padrão em que objetos com silhuetas menos definidas e cores semelhantes às do fundo, como os alimentos, exigem limiares mais flexíveis para

que não ocorra uma perda grande de informações importantes.

3.3.3 Teste com Imagens Externas

Para validar a capacidade de generalização do modelo, o algoritmo também foi testado em imagens externas não pertencentes ao dataset original da disciplina.

Na Figura 3, a aplicação do modelo em um ambiente interno com um grupo de amigos nos mostra que o impacto da variabilidade da confiança e do IoU foi menor em relação às Figuras 1 e 2. Isso demonstra uma característica de "estabilidade de classe", pois, nos três primeiros cenários, mesmo com as alterações dos limiares, a contagem da quantidade de pessoas permanece estável em 9 detecções. Isso indica que, para os indivíduos em primeiro e segundo plano, o modelo possui uma convicção sólida, e a sobreposição física entre os rostos e corpos não são suficientes para que o NMS descarte as detecções distintas.

Figura 3: Impacto dos limiares em ambiente com um grupo de pessoas.

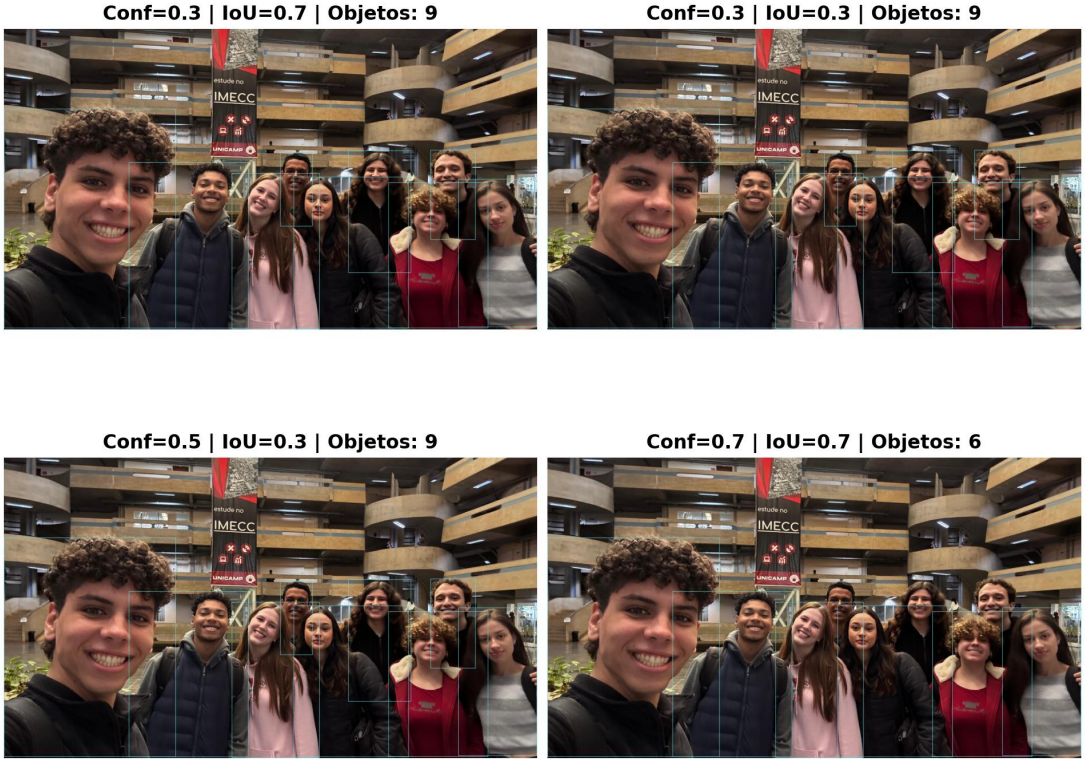


Tabela 3: Comparação de limiares em grupo de pessoas

Conf	IoU	N. Obj	Classes Detectadas
0.3	0.7	9	person (x9)
0.3	0.3	9	person (x9)
0.5	0.3	9	person (x9)
0.7	0.7	6	person (x6)

Nesta situação, o descarte somente foi feito ao elevar o limiar de confiança e o IoU para 0.7, reduzindo para 6 as pessoas identificadas. Uma observação importante para esta imagem é que as detecções suprimidas correspondem justamente às pessoas localizadas mais ao fundo ou que apresentam maior oclusão parcial quando submetidas à exigência de certeza máxima.

Como última análise, vemos que a imagem da praia (Figura 4) apresenta um desafio de oclusão e densidade, onde o comportamento do modelo reforça a dependência do limiar de confiança para a filtragem de ruído em escalas reduzidas. Com a confiança em 0.3, o modelo mantém apenas 13 detecções independentemente da variação do IoU entre 0.3 e 0.7. Esse fenômeno indica que, o modelo possui dificuldade de identificar muitas pessoas amontoadas num mesmo lugar.

Figura 4: Impacto dos limiares em ambiente com multidão



Tabela 4: Comparação de limiares em multidão na praia

Conf	IoU	N. Obj	Classes Detectadas
0.3	0.7	13	person (×11), umbrella (×2)
0.3	0.3	13	person (×11), umbrella (×2)
0.5	0.3	8	person (×8)
0.7	0.7	4	person (×4)

Com a transição para limiares de confiança mais altos (0.5 e 0.7) temos uma perda progressiva na contagem de objetos, reduzindo a contagem para 8 e, finalmente, 4 detecções. Resumidamente, nota-se que o modelo perde prioridade sobre as figuras humanas mais distantes e com menor contraste em relação ao fundo, mantendo apenas os indivíduos em primeiro plano e com silhuetas mais nítidas. Esta análise confirma que, em ambientes superlotados, a confiança do modelo é muito afetada pela escala dos objetos e pela complexidade do cenário, sem contar que adicionar um limiar excessivamente alto pode resultar em uma subcontagem significativa da população presente na imagem, principalmente esta que já foi iniciada com uma contagem baixa.

4 Discussão

4.1 Qualidade Geral e Papel dos Limiares

O modelo obteve excelente desempenho para objetos claros (confiança $> 85\%$). O limiar de confiança mostrou ser o controlador do *trade-off* entre precisão e recall, onde 0.5 apresentou o melhor equilíbrio prático. O IoU no NMS controlou a supressão de duplicatas: limiares altos (0.7) permitiram detecções muito sobrepostas (como garrafas próximas em *wine.jpg*), enquanto limiares baixos limparam melhor as predições de um mesmo alvo.

4.2 Comportamento do NMS por Classe

A implementação correta do NMS separando por *class_ids* provou-se essencial. Em imagens contendo objetos de classes distintas altamente sobrepostos (ex: uma pessoa montada num cavalo ou uma bicicleta na frente de um carro), o processamento independente evitou que a predição de maior confiança de uma classe suprimisse a detecção correta da outra classe no mesmo espaço espacial.

4.3 Generalização em Imagens Externas

Ao testar as imagens capturadas fora da base de dados original, notou-se que o modelo manteve robustez na detecção de classes do dia a dia (carros, pessoas, eletrônicos), validando que não houve *overfitting* restrito às características visuais (iluminação, resolução) das imagens padrão do projeto. Quedas de confiança nas imagens externas ocorreram apenas sob condições de baixa luminosidade ou ângulos atípicos de câmera.

4.4 Limitações e Propostas de Melhorias

Embora o modelo YOLO v11n apresente um desempenho excepcional em termos de latência e eficiência computacional, a análise experimental revelou gargalos inerentes à arquitetura compacta (*Nano*) e ao processamento padrão. Abaixo, discutem-se as limitações observadas e as respectivas propostas de aprimoramento técnico:

- **Escalabilidade da Arquitetura:** O uso da versão *Nano* (2.5M de parâmetros) impõe um limite de capacidade representacional. Em cenários de alta complexidade, como a imagem *city_ scene.jpg*, observou-se a omissão de objetos em escalas muito reduzidas ou sob oclusão parcial. Uma melhoria direta seria a migração para variantes mais robustas, como o **YOLO v11m** ou **v11l**, que possuem *backbones* mais profundos e maior capacidade de extração de características semânticas, ainda mantendo viabilidade para tempo real.

- **Refinamento da Métrica de Localização (IoU):** A implementação atual utiliza o IoU clássico (Interseção sobre União). Uma melhoria significativa seria a adoção de métricas evoluídas como o **DIoU (Distance-IoU)** ou **CIoU (Complete-IoU)**. Essas variantes consideram, além da sobreposição, a distância entre os centros das caixas e a proporção de aspecto, o que reduz erros de localização em objetos adjacentes e acelera a convergência do modelo.

- **Otimização do Pós-processamento (Soft-NMS e WBF):** O algoritmo *Non-Max Suppression* implementado atua de forma binária: se o IoU ultrapassa o limiar, a caixa é descartada. Em cenários de alta densidade (ex: multidões na praia), isso pode gerar falsos negativos. Propõe-se a substituição pelo **Soft-NMS**, que reduz a confiança de caixas sobrepostas em vez de eliminá-las sumariamente, ou pelo **Weighted Boxes Fusion (WBF)**, que realiza uma média ponderada das coordenadas de múltiplas predições, gerando *bounding boxes* mais estáveis e precisas.

- **Técnicas de Robustez (TTA e Augmentation):** Para mitigar falhas sob variações de iluminação e ângulos atípicos, poderia ser implementado o **Test-Time Augmentation (TTA)**. O TTA consiste em realizar múltiplas predições sobre a mesma imagem (rotacionada ou espelhada) e combinar os resultados, aumentando a confiança do sistema em ambientes não controlados.

- **Adaptação de Domínio (Fine-tuning):** O modelo utilizado foi pré-treinado no dataset COCO. Para aplicações específicas, como monitoramento industrial ou médico, o **ajuste fino (fine-tuning)** em um dataset proprietário anotado permitiria que a rede aprendesse características contextuais específicas, reduzindo a taxa de erro em classes que possuem alta variância intra-classe.

5 Referências

1. Redmon, J., et al. (2016). *You Only Look Once: Unified, Real-Time Object Detection.* CVPR.

2. Jocher, G., et al. (2024). *Ultralytics YOLO11.* Docs Ultralytics.

3. Florindo, J. (2025). *Semana 12 — Detecção de Objetos.* Material de aula, UNICAMP.

4. Lin, T. Y., et al. (2014). *Microsoft COCO: Common Objects in Context.* ECCV.